

昆仑1通讯库（KCL）设计文档

📘 本页面由边俊柯于2022/06/17迁移自wiki 陈庆澍空间

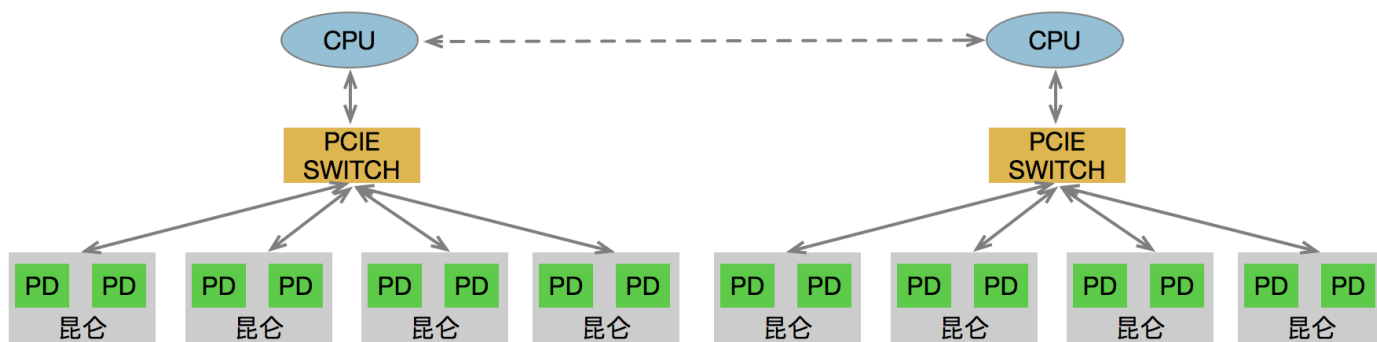
1、概述

昆仑一为加速深度学习计算专门设计的一款芯片，昆仑一支持预测和训练。为加速训练速度，训练过程经常会多卡一同训练，每张卡训练一个mini batch，中间要求进行梯度同步。目前GPU多卡训练时使用NCCL通讯库来进行梯度同步。为支持多卡训练，昆仑一也要求实现一套类似NCCL的通讯库，即昆仑通讯库（Kunlun Communication Library, abbr KCL as "cool"）。

2、昆仑一简介

2.1 硬件互连

昆仑一一种部署方式如图一所示，每个服务器上有8个昆仑芯片，每4个昆仑芯片连接一个PCI SWITCH，跨PCIE SWITCH的昆仑如果要通讯需要走CPU的通路（也有可能两个PCIE SWITCH同时连接到同一个PCIE SWITCH，这样子任意两个昆仑都能通过PCIE进行数据通讯）。每个昆仑内部有两个PD，每个PD拥有自己独立的计算资源和内存。在训练的过程中，每个PD独立处理一个mini batch的数据，PD与PD之间也需要进行梯度同步。



图一：昆仑一部署图

2.2 数据拷贝接口

昆仑一没有设计专用的数据通路用于昆仑与昆仑之间进行数据交互，昆仑与昆仑之间的数据拷贝只能通过host发起的PCIE DMA完成。昆仑一PCIE DMA可以参

考：<http://agroup.baidu.com/hanjinchen/md/article/1455522>

同一个昆仑内部的PD与PD之间支持SSE DMA，SSE DMA由XPU Cluster发起，具体的接口可以见Demo：http://icode.baidu.com/repos/baidu/xpu/api/blob/master:src/kernel/cpp/verify/verify_sse_pd_2_pd_dma.cpp

2.3 昆仑编程接口

昆仑芯片有两个核心的计算模块SD-CDNN和XPU-CLUSTER。SD-CDNN主要负责矩阵乘法运算，矩阵乘法后面可以接激活操作。XPU-CLUSTER支持向量的操作，如常见的加减乘除等。更多编程详情可以参考：http://xpu.baidu-int.com/doc/api/doc/01_basic.html。昆仑一通讯库实现过程中要求对不同PD上的数据进行累加操作，该部分功能可以放在XPU-CLUSTER上面实现。

3、概要设计

3.1 需求描述

昆仑一通讯库要求功能上对齐NCCL通讯库，具体包括：

- 支持all_reduce, reduce, broadcast操作，其中all-reduce的优先级最高，其他操作如果有需要可以继续添加；
- 支持单机和多机版本的collective communication，暂时可以先支持单机版本，多机版本后期再添加
- 通讯库接口要求简洁易用
- 通讯库要求挖缺硬件潜力，获取更好的性能
- 开发tools来诊断故障以及修复故障等，以及开发Regression/Stress/Reliability测试工具

3.2 接口说明

</>

C++ | 收起 ^

```
1 // Kcl是Kunlun Communication Library的缩写
2 // 该函数获取一个唯一的ID用于通讯库的初始化，所有参与通讯的PD，只需要一个PD的线程上调用就可以了
3 KclResult getId(KclUniqueId* id);
4
5
6 // 初始化通讯库
7 // ctx: 进行通讯时需要的结构体，该函数会初始化这个结构体
8 // rank: 当前PD的编号，编号是逻辑上的编号，所有参与通讯的PD从0到n-1编号，n为参与通讯的PD总数
9 // nranks: 参与通讯的PD总数
10 // id: getId获取的id，参与通讯的PD需要用同一个id初始化
11 KclResult initRank(KclContext* ctx, int rank, int nranks, const KclUniqueId* id);
12
```

```
13
14 // all reduce函数
15 // ctx: 通讯库的context, 在initRank中初始化, ctx包含了本rank的src和dst的device id
16 // sendbuf: 需要进行all reduce的数据地址
17 // recvbuf: all reduce后的结果存放地址
18 // count: sendbuf的数据量大小
19 // datatype: sendbuf和recvbuf中数据类型, 如float、bfloat、int16等
20 // op: all reduce中进行的操作, 比如sum, prod, min, max等
21 // stream: all reduce的kernel放在哪个stream上面执行(待定: 看是否放到ctx中, 因为
    buffer放在ctx中,
22 // 同一个ctx下不同的collective communication是不能并行的,
23 // 坏处是如果collective communication依赖其他kernel的话, 无法实现)
24 KclResult allReduce(const KclContext* ctx, const void* sendbuf, void*
    recvbuf, size_t count,
25                     KclDataType datatype, KclOp op, XPUStream stream)
26
27
28 // broadcast函数
29 // ctx: 通讯库的context, 在initRank中初始化
30 // sendbuf: 需要进行broadcast的数据地址
31 // recvbuf: broadcast后的结果存放地址
32 // count: sendbuf的数据量大小
33 // datatype: sendbuf和recvbuf中数据类型, 如float、bfloat、int16等
34 // root: 哪个rank作为broadcast的发起者
35 // stream: broadcast的kernel放在哪个stream上面执行(待定: 看是否放到ctx中, 因为buffer
    放在ctx中,
36 // 同一个ctx下不同的collective communication是不能并行的)
37 KclResult broadcast(const KclContext* ctx, const void* sendbuf, void*
    recvbuf, size_t count,
38                     KclDataType datatype, int root, XPUStream stream)
39
40
41 // reduce函数
42 // ctx: 通讯库的context, 在initRank中初始化
43 // sendbuf: 需要进行reduce的数据地址
44 // recvbuf: reduce后的结果存放地址
45 // count: sendbuf的数据量大小
46 // datatype: sendbuf和recvbuf中数据类型, 如float、bfloat、int16等
47 // root: 哪个rank作为reduce的数据的接收者
48 // stream: reduce的kernel放在哪个stream上面执行(待定: 看是否放到ctx中, 因为buffer放在
    ctx中,
49 // 同一个ctx下不同的collective communication是不能并行的)
50 KclResult reduce(const KclContext* ctx, const void* sendbuf, void* recvbuf,
    size_t count,
```

3.3 通讯算法

collective communication操作针对不同的互连结构，可以选择不同的算法，如ring、tree、2d-torus等等（[all-reduce算法](#)）。昆仑一通讯库优先支持单机多卡的通讯，由于单机情况卡数比较少，且ring算法带宽最优、实现相对简单，单机多卡情况下计划使用ring算法。多机情况下，卡的数量增多，每次通讯的overhead增大，可以考虑2d-torus算法。

3.4 拓扑检测

在使用ring或者2d-torus算法时，需要将所有的昆仑构成一个ring或者构成横向纵向的多个ring。由于昆仑和昆仑之间可能通过PCIe Switch、PCIe Host Bridge、Network互连，且不同互连进行通讯的开销不同，因为需要知道昆仑与昆仑之间的互连拓扑，从而能构建一个最优的ring，保证通讯的开销最低。GPU提供了nvidia-smi的命令行工具可以检测GPU与GPU之间的互连拓扑（如图二所示）。图二的4个GPU想构成一个ring时，最佳选择是GPU0与GPU1靠近，GPU2与GPU3靠近，因为GPU0与GPU1连在同一个PCIe switch上，GPU2和GPU3连在一个PCIe switch上。一种可能的构成ring的方式是0->1->2->3->0。

在实现昆仑一通讯库时，也依赖驱动或lspci或其他工具提供底层硬件互连拓扑，通讯库根据硬件互连拓扑，决定软件层面ring、2d-torus或其他软件拓扑的构建。

```
204723148.job-0bb5cde66c1abd8e-trainer ~ # nvidia-smi topo -m
  GPU0   GPU1   GPU2   GPU3   mlx5_0  CPU Affinity
GPU0     X     PIX     PXB     PXB     SOC      0-13
GPU1     PIX    X     PXB     PXB     SOC      0-13
GPU2     PXB    PXB    X     PIX     SOC      0-13
GPU3     PXB    PXB    PIX    X     SOC      0-13
mlx5_0   SOC     SOC     SOC     SOC      X

Legend:
 X      = Self
 SOC    = Connection traversing PCIe as well as the SMP link between CPU sockets(e.g. QPI)
 PHB    = Connection traversing PCIe as well as a PCIe Host Bridge (typically the CPU)
 PXB    = Connection traversing multiple PCIe switches (without traversing the PCIe Host Bridge)
 PIX    = Connection traversing a single PCIe switch
 NV#    = Connection traversing a bonded set of # NVLinks
204723148.job-0bb5cde66c1abd8e-trainer ~ #
```

图二：GPU拓扑检测

3.5 框架集成

KCL主要针对深度学习场景，需要集成到Paddle等深度学习框架中。Paddle目前将通讯作为一个op插入到训练场景中，只需要在相应的通讯op中调用KCL的API就可以进行梯度同步。

3.6 测试接口

KCL需要提供一套单测、性能测试（带宽利用率和latency）、回归测试等接口。

4、详细设计

4.1 拓扑检测

昆仑一芯片对外暴露的接口为device_id, 同一个芯片内部两个PD拥有不同的device_id。在昆仑与昆仑之间的进行通讯时，优先级为：

1. 同一个芯片不同PD
2. 挂载在同一个PCIe Switch下的不同PD
3. 通过多个PCIe Switch能通讯的PD
4. 挂载在同一个CPU下的不同PD
5. 挂载在同一个服务器上的不同PD
6. 不同服务器中的PD

linux提供lspci的工具通过bdf查询查询pci设备信息（如图三所示）。以图三42:00.0设备为例，42:00.0设备是一个NVIDIA的gpu，它和43:00.0, 44:00.0, 45:00.0连接在3f:00.0上面，lspci -s 3f:00.0可以查看3f:00.0设备的详细信息，如图四所示。图四第一行10b5表示厂商id，9797表示设备id，在<https://pci-ids.ucw.cz/>上面可以查询到具体设备的信息，比如PCIe Switch, Bridge等。10b5:9797是一款PCIe switch，所以42:00.0, 43:00.0, 44:00.0, 45:00.0这几款设备通过PCIe Switch互连。同样的方法也可以知道其他设备是通过什么方式相连。如图五显示83:00.0和84:00.0两个GPU通过PCIe Bridge互连，均挂载在0号总线上。

为实现上述拓扑检测，需要**驱动提供方法查询pd对应的bdf**，如果两个PD拥有相同的device_id，表示设备在同一个昆仑芯片上。获取了昆仑PD的互连拓扑之后，软件算法根据优先级设定，构建一个软件ring。

通过lspci获取PCIe 设备信息需要有接口，更多内容可以参考：<https://github.com/pciutils/pciutils>

```
+-[0000:3a]--+00.0-[3b-4c]---00.0-[3c-4c]--+08.0-[3d]--
|                                     +-0c.0-[3e]--
|                                     +-10.0-[3f-45]----00.0-[40-45]--+00.0-[41]--
|                                     |
|                                     +-08.0-[42]----00.0  NVIDIA Corporation Device 1b38
|                                     +-0c.0-[43]----00.0  NVIDIA Corporation Device 1b38
|                                     +-10.0-[44]----00.0  NVIDIA Corporation Device 1b38
|                                     \-14.0-[45]----00.0  NVIDIA Corporation Device 1b38
|                                     \-14.0-[46-4c]----00.0-[47-4c]--+00.0-[48]--
|                                     |
|                                     +-08.0-[49]----00.0  NVIDIA Corporation Device 1b38
|                                     +-0c.0-[4a]----00.0  NVIDIA Corporation Device 1b38
|                                     +-10.0-[4b]----00.0  NVIDIA Corporation Device 1b38
|                                     \-14.0-[4c]----00.0  NVIDIA Corporation Device 1b38
```

图三： lspci 树形结构结果


```
1022689473.job-0bb5d09b444011d5-trainer 47 # lspci -s 3f:00.0 -vv -n
3f:00.0 0604: 10b5:9797 (rev b0) (prog-if 00 [Normal decode])
    Physical Slot: 16
    Control: I/O+ Mem+ BusMaster+ SpecCycle- MemWINV- VGASnoop- ParErr+ Stepping- SERR+ FastB2B- DisINTx-
    Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
    Latency: 0, Cache Line Size: 64 bytes
    Region 0: Memory at b8000000 (32-bit, non-prefetchable) [size=256K]
    Bus: primary=3f, secondary=40, subordinate=45, sec-latency=0
    Memory behind bridge: b4000000-b7ffffff
    Prefetchable memory behind bridge: 00003ac000000000-00003af801ffffff
    Secondary status: 66MHz- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- <SERR- <PERR-
    BridgeCtl: Parity+ SERR+ NoISA- VGA- MAbort- >Reset- FastB2B-
        PriDiscTmr- SecDiscTmr- DiscTmrStat- DiscTmrSRERen-
    Capabilities: [40] Power Management version 3
        Flags: PMEClk- DSI- D1- D2- AuxCurrent=0mA PME(D0+,D1-,D2-,D3hot+,D3cold+)
        Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
    Capabilities: [48] MSI: Enable- Count=1/8 Maskable+ 64bit+
        Address: 0000000000000000 Data: 0000
        Masking: 00000000 Pending: 00000000
    Capabilities: [68] Express (v2) Upstream Port, MSI 00
        DevCap: MaxPayload 2048 bytes, PhantFunc 0, Latency L0s <64ns, L1 <1us
            ExtTag- AttnBtn- AttnInd- PwrInd- RBE+ FLReset-SlotPowerLimit 25.000W
        DevCtl: Report errors: Correctable- Non-Fatal+ Fatal+ Unsupported-
            RlxdOrd+ ExtTag- PhantFunc- AuxPwr- NoSnoop+
            MaxPayload 256 bytes, MaxReadReq 128 bytes
        DevSta: CorrErr+ UncorrErr- FatalErr- UnsuppReq+ AuxPwr- TransPend-
        LnkCap: Port #4, Speed 8GT/s, Width x16, ASPM L1, Latency L0 <4us, L1 <4us
            ClockPM- Surprise- LLActRep- BwNot-
```

图四: lspci 具体设备信息

```
+-[0000:80]--00.0-[81]----00.0 LSI Logic / Symbios Logic MegaRAID SAS 2108 [Liberator]
|      +-01.0-[82]----00.0 Mellanox Technologies MT27520 Family
|      +-02.0-[83]----00.0 NVIDIA Corporation Device 1023
|      +-03.0-[84]----00.0 NVIDIA Corporation Device 1023
```

图五: lspci两个设备通过PCIe bridge互连

4.2 ring all reduce

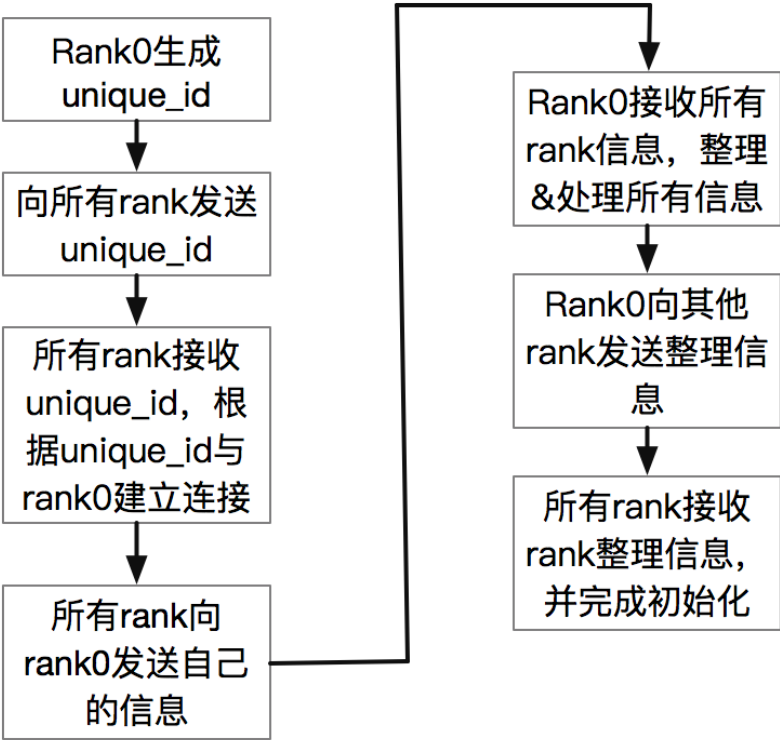
KCL使用ring算法实现all reduce操作。ring all-reduce把所有的节点串成一个环，算法的进行分两步：第一步，scatter-reduce；第二步，allgather。在第一步中，昆仑PD 将交换数据，使得每个 PD 最终都有一个最终结果的数据块。在第二步中，GPU 将交换那些块，使得所有 PD最终得到完整的最后结果，更多细节参考：[all-reduce算法](#)。

假设p个PD需要进行梯度同步，则需要 $2(p-1)$ 个步骤才能完成完整的all reduce操作，前p-1步骤中包含获取前一个节点的数据，累加两个细小的操作，且必须保证已经获取了前一个节点的数据后，才能进行累加操作。由于昆仑一不同PD间数据传输只能在Host端发起，因此将数据传输和累加op的控制放在Host端。对于某个PD，Host先发起一个Memcpy的请求，等待Memcpy完成后，Host下发累加kernel，等累加完成后，继续Memcpy，然后累加，不断重复该过程，知道完成all reduce操作。该方案的问题在于Memcpy是同步操作，当Memcpy完成后，再下发累加kernel，累加kernel的overhead在微秒级别，且一次完整的all reduce需要p-1次累加操作，累加的overhead过大。

一种优化方案是使用binary tree算法（参考：[all-reduce算法](#)），该方案只需要调用 $\lg(p)$ 次累加和memcpy。

4.3 初始化

KCL初始化时，每个昆仑PD只知道自己的信息，为了完成通讯，PD与PD之间需要交换一些元信息，比如PD的device id， bdf， PD与PD之间的互连信息等等。初始化过程中，选取rank 0生成一个unique_id且rank0监听在某个网络端口中等待其他PD连接，rank 0生成的unique_id包含了监听端口的信息，其他PD可以根据该unique_id与rank 0通讯。rank 0生成unique id后，发送给所有PD（该部分需要用户程序自己实现）。各PD收到unique_id之后，解析unique_id后通过TCP/IP连接上rank 0，并且自身监听在某个端口上，各PD把自己监听的端口号信息发送给rank 0，同时把自身的元数据发送给rank 0。rank 0接收所有PD的元信息后，汇总并处理，比如生成ring信息，最后发给所有的PD。各PD收到汇总和处理后的信息进行初始化。



图六： 初始化流程图

4.4 Mock接口

KCL开发过程中，由于硬件没有ready，模拟器又不支持模拟多个昆仑卡，因此在开发过程中需要Mock多个接口，包括内存拷贝、拓扑检测接口等等。KCL开发过程中，先基于Mock接口测试程序的正确性，并且开发单测和其他测试，等硬件或者模拟器ready后，再进行系统的测试。

4.5 与昆仑二兼容

昆仑二通讯库的调度、同步都放在了芯片内部，而昆仑一的调度同步则放在Host端，两个通讯库很难实现兼容。所以，昆仑二计划新开一个分支，并且复用昆仑一API和初始化部分，底层调度、同步和kernel全部重写。

5、 昆仑通讯库依赖

2023/7/19 17:18

昆仑1通讯库（KCL）设计文档

1	模块	接口	备注
2	driver&runtime	bool set_device(int device_id)	调用KCL进行初始化时，需要设置device id，接口中传入device id
3	driver&runtime	string get_device_bdf(int device_id)	KCL通过lspci等工具获取互连拓扑时，需要知道设备id
4	driver&runtime	bool kunlun_memcpy(int src_id, void* str_ptr, int dst_id, void* dst, int size)	昆仑芯片与芯片之前的P2P拷贝接口
5	模拟器	多设备支持	模拟器需要能模拟多个设备，用于验证KCL的正确性
6	模拟器	昆仑与昆仑之间数据拷贝	模拟器需要能模拟昆仑与昆仑之间数据拷贝功能，验证KCL的正确性
7	昆仑一芯片	真实的硬件	真实环境验证KCL正确性和性能调优

6、开发计划

计划先开发一个支持单机、3个通讯接口的昆仑通讯库，共计需要单人30个工作日的开发量，具体如下：

1	模块	描述	工作量(单人多少工作日)	备注
2	接口设计	设计完整的交互接口	2	先考虑broadcast、reduce、all reduce这三个通讯接口，剩下接口后期再加
3	init	获取昆仑参与通讯的昆仑信息，并且建立信息的通讯机制（基于tcp）	5	
4	拓扑检测	识别昆仑芯片互连拓扑，并且建立ring	10	先实现单机版本
5	broadcast	broadcast api	4	
6	reduce	reduce api	3	
7	all reduce	all reduce api	3	
8	测试单元	基础的性能和正确性验证接口	3	

